ÓRBITA

README Técnico

Visão geral de arquitetura, stack proposta e estrutura do projeto

Autoria: Geovanna Tamagusko | Documento gerado em 19/10/2026
Status do projeto: planejamento — sem código implementado até o momento

1. Visão geral

O Órbita é um aplicativo pessoal de gerenciamento de projetos, inicialmente de uso individual, com acesso autenticado e sincronização entre dispositivos. O usuário visualiza uma lista de projetos e, ao selecionar um deles, acessa detalhes como prioridade, lista de tarefas (to-dos) em formato de checkbox, descrição, data final, temporizador pomodoro, gráficos de acompanhamento e notificações/alarmes.

Este documento descreve a arquitetura técnica proposta. Não há código implementado até o momento — o projeto está em fase de planejamento.

---

2. Escopo e requisito central de arquitetura

O requisito que mais define a arquitetura é o acesso multi-dispositivo com conta: os dados precisam estar disponíveis em qualquer aparelho em que o usuário fizer login, não apenas no dispositivo onde foram criados. Isso descarta armazenamento puramente local (ex.: apenas localStorage do navegador ou um banco de dados SQLite isolado na máquina) como solução final — é necessário um backend com banco de dados na nuvem e autenticação.

3. Stack

Front-end confirmado como HTML/CSS/JS e backend confirmado como Firebase, rodando na conta Firebase pessoal da autora. O comparativo abaixo documenta a análise que levou à escolha.

3.1. Front-end

- HTML, CSS e JavaScript puro (conforme preferência já manifestada pelo usuário).
- Estruturado como PWA (Progressive Web App) — necessário para viabilizar notificações/alarmes e permitir instalação do app no celular.

3.2. Backend e banco de dados

Duas opções comparáveis para quem não tem experiência em administração de banco de dados:

Critério	Firebase (Firestore)	Supabase (Postgres)
Curva de aprendizado	Mais simples, no-code na config inicial	Exige SQL básico
Autenticação pronta	Sim (Firebase Auth)	Sim (Supabase Auth)
Modelo de dados	Documentos (NoSQL)	Relacional (tabelas)
Plano gratuito	Sim, generoso para uso pessoal	Sim, generoso para uso pessoal
Notificações push	Integração nativa (Firebase Cloud Messaging)	Requer serviço externo
Indicação	Melhor para começar rápido sem saber SQL	Melhor se quiser aprender SQL junto

Decisão: Firebase, pela simplicidade de configuração e por já incluir notificações push nativas — relevante para o requisito de alarme/notificação do app. Quem quiser rodar sua própria cópia do projeto deve criar um projeto Firebase gratuito próprio e substituir as chaves de configuração pelas suas — não há necessidade de suportar outro banco de dados para isso.

3.3. Notificações e alarmes — limitação técnica importante

Um site aberto em uma aba comum de navegador não consegue disparar um alarme se a aba estiver fechada ou o navegador não estiver aberto. Para que o Órbita envie notificações reais (tipo celular vibrando, mesmo com o app fechado), é necessário estruturar o front-end como PWA com Service Worker e usar um serviço de push (ex.: Firebase Cloud Messaging). Sem isso, só é possível mostrar alertas enquanto o app estiver aberto na tela.

3.4. Visual e navegação

- Tema escuro como padrão (fundo quase preto, texto claro, acento roxo/violeta).
- Navegação em painel: lista de projetos fixa à esquerda, detalhe do projeto abre ao lado (sem troca de tela cheia).
- Mockup navegável das telas disponível em arquivo próprio (ver docs/mockup.md).

4. Estrutura de pastas sugerida

```
orbita/
  ■■■ index.html
  ■■■ /css
  ■   ■■■ estilo.css
  ■■■ /js
  ■   ■■■ app.js (lógica principal)
  ■   ■■■ firebase-config.js (conexão com o backend)
  ■   ■■■ pomodoro.js
  ■   ■■■ graficos.js
  ■   ■■■ notificacoes.js
  ■■■ service-worker.js (necessário para PWA/notificações)
  ■■■ manifest.json (necessário para instalação como PWA)
  ■■■ /assets
```

5. Como rodar (quando o código existir)

- Criar projeto no console do Firebase e ativar Authentication + Firestore.
- Copiar as chaves de configuração para js/firebase-config.js.
- Servir os arquivos estáticos com um servidor local (ex.: Live Server do VS Code) — PWAs exigem HTTPS ou localhost para funcionar corretamente, inclusive para testar o Service Worker.
- Testar login, criação de projeto e sincronização entre duas abas/dispositivos diferentes.

6. Roadmap sugerido

- Etapa 1 — Estrutura estática: telas de lista de projetos e detalhe do projeto, sem backend.

- Etapa 2 — Autenticação e persistência: conectar ao Firebase, salvar e carregar projetos por usuário.
- Etapa 3 — Pomodoro e gráficos: funcionalidades internas da tela de detalhe.
- Etapa 4 — PWA e notificações: manifest, service worker e alarmes.
- Etapa 5 — Sistema de recompensa: XP, streak e tela do planeta.
- Etapa 6 — Refino visual e responsividade (uso em celular).